DeepShield - AI-Powered Deepfake Detection System

Course: PROJECT

Course Code: CAP599

Group No: D2523

Submitted by:

Prashant Patil (12527088)
Sachin Jangra (12500231)
Thaneshwar Chawle (12500566)
Kileshwar (12518653)

Submitted to:

Mr. Gagandeep Sethi
UID: 31878

Lovely Professional University,

School of Computer Applications,

Punjab

Table of Content

Contents

1. Introduction 3

2. Problem Statement 3

3. Objectives 3

4. Methodology 4

5. Flow Diagram of Methodology 5

6. Deployment 6

7. Flow Diagram of Methodology 6

8. Team & Technology Responsibilities 7

9. Conclusion 7

Synopsis

1. Introduction

The creation of deep fakes happens through artificial intelligence. It helps in creating realistic images or videos of something that never occurred. This technology has so much creativity in terms of the opportunities it provides, but at the same time, it has raised very important issues, especially in terms of cybersecurity and privacy. This capability, if utilized in an unethical manner, can lead to the spread of false information, creation of fake news, and even tarnishing reputations. This, in itself, speaks volumes about how there is a need to mitigate risks, both from an ethical and security perspective, related to this tool. Hence, we believe that there can be an initiation of a detection system related to web-hosted deep fakes that can be made through machine learning algorithms. It can be achieved through various algorithms related to manipulation detection, frame-by-frame analysis of images and videos, in order to let fair detection occur.

2. Problem Statement

The ever-improving accessibility and advancement of deepfake technology have brought about the challenge of distinguishing between the genuine and the manipulated version of the content. The extent of the harm that the technology could cause has also increased and now includes privacy violations and the spreading of misinformation. There is no detection method that has been fully effective in the detection of the content produced by the technology since the methods of creating the technology continue to advance. There is the need to develop an effective detection method that will be able to differentiate between the deepfake content and the various methods of creating it within the same platform. This project seeks to solve the problem by using advanced deep learning technology in the detection of image forgery and training the model on genuine and deepfake objects

3. Objectives

- Creating a Dataset: Collecting a dataset of genuine as well as deepfake images
- and videos for training as well as testing the model.
- Training the Model: Developing a deep learning CNN model by applying
- machine learning concepts to detect deepfake images.
- Evaluation of the Model: Evaluating the model based on various accuracy
- parameters as well as proficiency of the model in various deepfake creation
- methods.
- 4. Implementing the System: Developing a web application or software that
- will allow users to use it for detecting deepfake images.

4. Methodology

1. Dataset Collection

The model is trained and tested on three major datasets, namely FaceForensics++, the Deepfake Detection Challenge dataset, and the Celeb-DF dataset.

2. Preprocessing

1. Resize Images & Videos: Images and videos are resized to a standard resolution for consistency.
2. Audio & Multiple Faces Removal: Videos with audio and videos with more than one face are removed to avoid errors during processing.
3. Splitting Videos into Frames: Videos are split into frames for further analysis.

3. Model Architecture

- Transfer Learning: Pre-trained models like ResNet (TensorFlow) and DeepFaceLab are employed to benefit from pre-trained features and minimize the training time.
- Custom CNN: A Convolutional Neural Network is developed and trained from scratch to perform well on the task of deepfake detection.

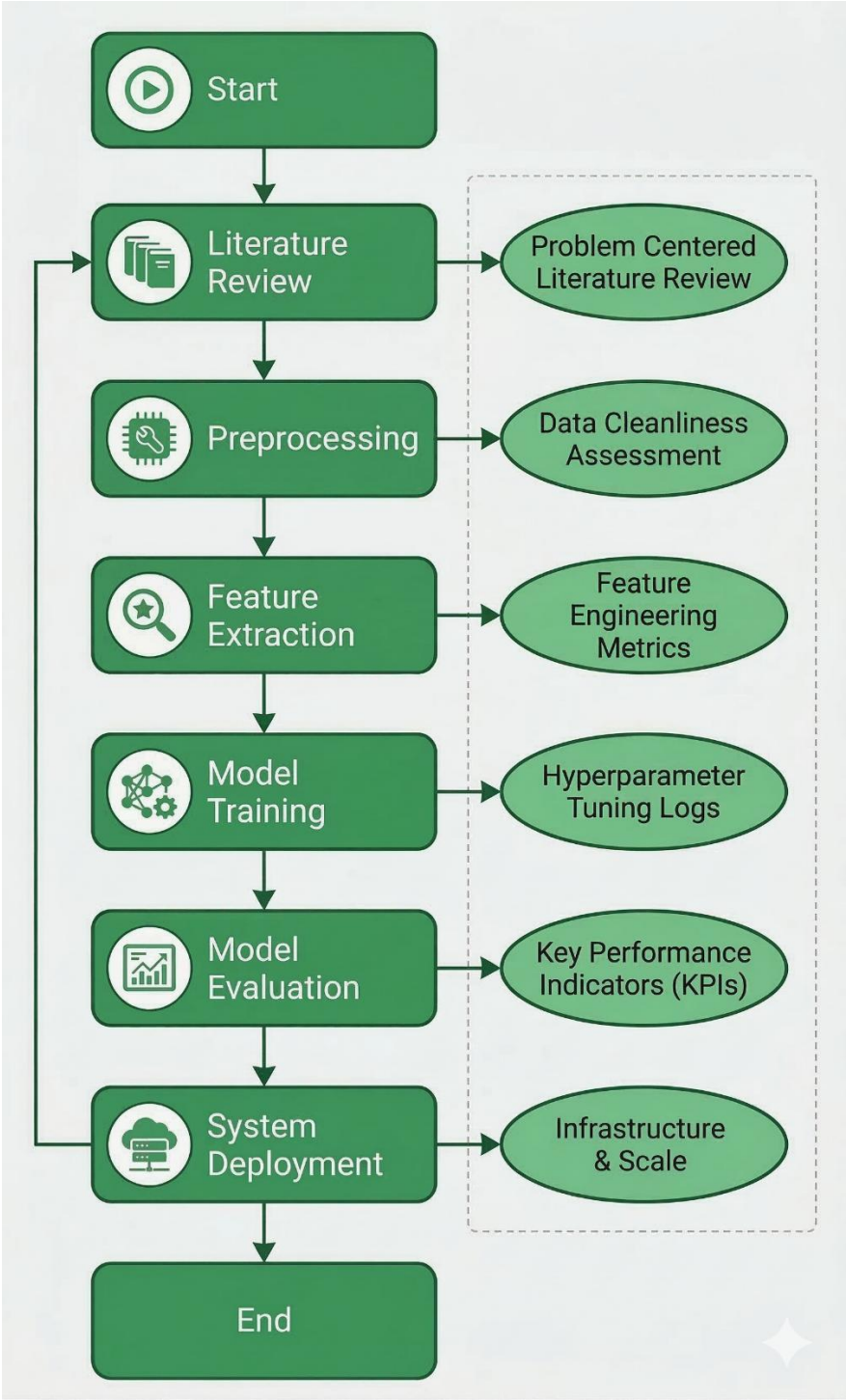
4. Training Strategy

- Data Split: The dataset is divided into training (70%), validation (15%), and testing sets (15%).
- Data Augmentation: Contrast enhancement, flipping, and adding noise to the data are employed to enhance the robustness of the model.
- Feature Extraction: The model is trained to learn the distinctive features between the original and the manipulated content of the videos.
- GPU Acceleration: Google Colab is employed to tune the model to its full potential and accelerate the training process.
- Optimization: Dropout and batch normalization techniques are employed to enhance the performance of the model and avoid overfitting.
- Testing & Validation: The model is tested on the data to evaluate its ability to generalize and perform well on unseen data.

5. Evaluation Metrics

1. Accuracy: The percentage of total predictions that the model has correctly classified out of the total number of predictions.
2. Precision: Out of the total number of videos classified as deepfake by the model, the percentage of videos that were actually deepfake

5. Flow Diagram of Methodology



6. Deployment

- Front-End: A clean and interactive front-end is built using HTML, CSS, and JavaScript. This will allow users to upload their images or videos and view the detection output in real time.
- Back-End: The application logic and the inference pipeline of the model will be built using a Python-based framework like Flask, FastAPI, or Django. This will expose the API endpoints that will be connected to the front-end and the trained model.
- Alternative Front-End: Streamlit and React could also be considered as alternatives to provide a more interactive and modern front-end UI/UX experience with less development overhead.
- Model Hosting: The trained model will be hosted on cloud environments like AWS, Google Cloud, and Heroku to ensure scalability and performance.

7. Flow Diagram of Methodology

Timeline

Month 1: Research & Data Preparation

Week	Tasks
Week 1	- Literature review - Tool selection (TensorFlow, Flask, etc.) - Environment setup
Week 2	- Dataset collection (FaceForensics++, DFDC, Celeb-DF)
Week 3	- Preprocessing (resizing, audio & multi-face removal, frame splitting)
Week 4	- Data augmentation (contrast enhancement, flipping, noise addition)

Month 2: Model Development & Training

Week	Tasks
Week 5	- Feature extraction strategy design and implementation
Week 6	- Transfer learning setup using ResNet / DeepFaceLab
Week 7	- Custom CNN design and training on Google Colab (GPU)
Week 8	- Hyperparameter tuning (dropout, batch normalization, learning rate)

Month 3: Evaluation, Deployment & Documentation

Week	Tasks
Week 9	- Model evaluation (Accuracy, Precision, Recall, F1-Score, AUC-ROC)
Week 10	- Re-training and fine-tuning based on evaluation results
Week 11	- Front-end (HTML/CSS/JS or React/Streamlit) + Back-end (Flask/FastAPI) development
Week 12	- Cloud deployment (AWS / Google Cloud / Heroku) - Final testing - Documentation

8. Team & Technology Responsibilities

Member	Role	Technologies / Tasks
Sachin	Back-End Developer	Flask, FastAPI/Django API development, model integration, server-side logic
Kileshwar	Back-End Developer	Database handling, endpoint development, deployment pipeline
Prashant Patil	Data & ML Engineer	Dataset collection (FaceForensics++, DFDC, Celeb-DF), preprocessing, model training, testing & evaluation
Tahneshwar	Front-End Developer	HTML, CSS, JavaScript, React/Streamlit — UI design

9. Conclusion

The problem of digital trust is compounded by the advent of deepfake technology. DeepShield addresses this problem by utilizing a transfer learning approach involving ResNet and a custom CNN, along with FaceForensics++, DFDC, and Celeb-DF datasets. The approach is assessed by utilizing Accuracy, Precision, Recall, F1-Score, and AUC-ROC metrics. The proposed system is accessible through a web interface that can be deployed on cloud services, thus showcasing the potential of machine learning for synthetic media detection.